

Cahier des Charges – Projet M1

Mastère Cybersécurité

I. Contexte et objectifs du projet

Le client est une société spécialisée en cybersécurité offensive, réalisant régulièrement des tests d'intrusion pour des entreprises privées et des institutions publiques.

Aujourd'hui, ces tests reposent majoritairement sur des manipulations manuelles, ce qui les rend longs et parfois hétérogènes selon les intervenants.

Objectif général :

Développer une toolbox automatisée permettant de réaliser efficacement l'ensemble des étapes d'un pentest, avec une interface simple, des modules réutilisables et un reporting standardisé.

Objectifs spécifiques :

- Réduire d'au moins 40 % le temps de réalisation d'un pentest ;
- Standardiser les pratiques internes ;
- Proposer une interface exploitable par des profils analystes, même peu développeurs ;
- Renforcer la qualité et la lisibilité des rapports ;
- Permettre une intégration simple dans l'écosystème technique du client.

II. Organisation de l'entreprise et enjeux par pôle

La toolbox devra être modulaire et permettre des usages adaptés à chaque pôle.

Pôle	Besoins spécifiques	Exemples d'outils intégrables
Sécurité (SOC, EDR, XDR)	Tests ciblés sur les systèmes de détection et les flux	Nmap, Metasploit, Wireshark, OWASP ZAP
Développement SaaS	Tests d'applications web et API	Burp Suite, Postman, SQLmap, Dependency-Check
Infrastructure	Évaluation des systèmes internes et réseaux	OpenVAS, Nessus, Hydra, Aircrack-ng
Support client	Tests de sécurité des outils de communication	Nikto, SSLyze, Ettercap, Maltego

III. Fonctionnalités principales attendues

- Modules automatisés couvrant toutes les étapes du pentest : reconnaissance, scan, exploitation, post-exploitation ;
- Intégration d'outils open-source via API, CLI ou bibliothèques Python ;
- Reporting automatisé, exportable et lisible ;
- Interface simple, utilisable sans compétences avancées en développement web ;
- Architecture modulaire, permettant l'ajout de futurs modules ;
- Sécurisation de l'outil lui-même (authentification, chiffrement, logging) ;
- (Bonus) : ajout d'un module forensique pour l'analyse post-compromission.

IV. Interface utilisateur adaptée

L'interface devra être simple, lisible et intuitive.

Aucune compétence avancée en frontend n'est requise. Un mode CLI est accepté si mieux adapté, tant que l'ergonomie reste lisible.

L'interface sera mise en place par vos soins exemple : HTML/CSS de base avec Flask + Jinja2.

V. Solution technique conseillée

La toolbox devra reposer sur une architecture modulaire, sécurisée et automatisée.

1. Architecture générale : Python 3.x, Flask ou FastAPI, Poetr
2. Modules fonctionnels : Nmap, OpenVAS/Nessus, Metasploit, scripts personnalisés
3. Intégration d'outils open-source : ZAP, SQLmap, Hydra, Aircrack-ng
4. Stockage & base de données : PostgreSQL, MinIO
5. Reporting : Jinja2, export PDF/HTML/CSV, D3.js ou Matplotlib
6. Orchestration : Celery, Redis
7. Sécurité : chiffrement Fernet, HTTPS, RBAC, audit logs
8. Conteneurisation : Docker, Docker Compose, CI/CD (GitLab)
9. Évolutivité : système de plugins, API documentée avec Swagger
10. Forensique (bonus) : Cuckoo Sandbox, Wireshark, ClamAV, VirusTotal API

Recommandations supplémentaires :

- Obfuscation, tamper-proofing, anti-injection
- Conformité RGPD et cadre éthique
- Gestion des mises à jour
- Intégration dans l'écosystème client
- KPIs : temps d'exécution, vulnérabilités détectées, stabilité
- Résilience, sauvegardes, évolution vers IA/ML

VI. Répartition des rôles et tâches

Exemple de répartition

- Étudiant 1 – Architecte / Développeur back-end : architecture, orchestration, intégration outils, sécurisation.
- Étudiant 2 – Analyste / Forensique / QA : intégration des outils de scan, module forensique, tests, validation.
- Étudiant 3 – Interface & Reporting : interface Flask, génération des rapports, tableaux de bord, doc utilisateur.

Tâches partagées : gestion de projet (Scrum), questions client, soutenance, documentation technique et fonctionnelle.

VII. Aide complémentaire et ressources conseillées

GitHub Student Developer Pack : accès à des outils professionnels (JetBrains, MongoDB Atlas, Canva...)

Outils recommandés :

- Gestion de projet : Trello, Jira, Notion
- Versioning & CI/CD : Git, GitLab CI, GitHub Actions
- Communication : Slack, Microsoft Teams, Discord
- Développement : Visual Studio Code, PyCharm
- Conteneurisation : Docker, Docker Compose
- Virtualisation : VirtualBox, VMware Workstation, Hyper-V

Bonnes pratiques :

- Respect du cadre légal et éthique (RGPD, autorisations)
- Documentation continue du projet
- Application des principes de sécurité, de propreté de code et de modularité
- Conception réutilisable et extensible de l'outil